

Lista de Exercícios 3 – Listas Encadeadas

1. Escrever uma função que recebe como entrada uma lista encadeada de inteiros e retorne a quantidade de elementos da lista.
2. Escreva uma função que procure um elemento n na lista simplesmente encadeada e retorne 1 (TRUE) caso ele esteja presente e 0 (FALSE) caso contrário.
3. Escreva uma função que procure um elemento n na lista simplesmente encadeada circular e retorne 1 (TRUE) caso ele esteja presente e 0 (FALSE) caso contrário.
4. Escrever uma função que recebe como entrada uma lista duplamente encadeada de inteiros e retorne a quantidade de elementos da lista.
5. Escrever uma função que recebe como entrada uma lista encadeada circular de inteiros e retorne a quantidade de elementos da lista.
6. Escrever uma função que receba como entrada uma lista simplesmente encadeada e ordene os elementos da lista em ordem crescente.
7. Dada uma lista encadeada de caracteres formada por uma sequência alternada de letras e dígitos, construa um método que retorne uma lista na qual as letras são mantidas na sequência original e os dígitos são colocados na ordem inversa. Exemplos:

A 1 E 5 T 7 W 8 G	→	A E T W G 8 7 5 1
3 C 9 H 4 Q 6	→	C H Q 6 4 9 3

Como mostram os exemplos, as letras devem ser mostradas primeiro, seguidas dos dígitos. Sugestão: usar uma fila e uma pilha.

8. Construa um método que recebe uma lista encadeada de números inteiros e retorna uma lista sem repetições, ou seja, uma lista onde cada número apareça apenas uma vez. Exemplo:

12 5 -7 8 5 9 12 1 8 → 12 5 -7 8 9 1

9. Implemente uma função que verifique se duas listas encadeadas são iguais. Duas listas são consideradas iguais se têm a mesma sequência de elementos. O protótipo da função deve ser dado por:

```
int igual (Lista* l1, Lista* l2);
```

10. Implemente uma função que crie uma cópia de uma lista encadeada. O protótipo da função deve ser dado por:

```
Lista* copia (Lista* l);
```

11. Escrever funções que recebem como entrada uma lista encadeada de inteiros e retornam:
 - a) O maior elemento da lista;
 - b) O menor elemento da lista;
 - c) O primeiro elemento da lista;
 - d) O endereço do primeiro elemento da lista;
 - e) O último elemento da lista;

- f) O endereço do último elemento da lista;
- g) O endereço do maior elemento da lista;

12. Considere as seguintes afirmativas:

- I. Numa fila as inserções são realizadas em um extremo e as remoções no outro extremo.
- II. A fila e a pilha são consideradas casos especiais da lista.
- III. Numa fila a inserção e a retirada são feitas no mesmo extremo.
- IV. Numa lista a inserção e a retirada podem ser feitas em qualquer posição.
- V. Numa pilha apenas a inserção pode ser feita em qualquer posição.

Quais são as afirmativas verdadeiras?

- a) Somente I e III
- b) Somente II, III e IV
- c) Somente I, II e IV
- d) Somente II, IV e V
- e) Todas as afirmativas

13. Desenvolver um programa que insere 10 números inteiros em uma lista inicialmente vazia. Em seguida, o programa deverá excluir os números ímpares existentes na lista deixando apenas os números pares.

14. Escrever uma função que receba como parâmetro a cabeça de uma lista encadeada de inteiros e retorne o último valor armazenado na lista ou -1 caso a lista esteja vazia. Considere a estrutura abaixo.

```
struct lista {
    int num;
    struct lista *next;
};
typedef struct lista Lista
```

15. Implemente uma função *inserir()* que realiza a inserção dos elementos de uma lista de inteiros ordenadamente. O protótipo da função deve ser dado por:

```
Lista *inserir (Lista* l, int nro);
```

16. Escreva uma função para executar cada uma das seguintes operações:

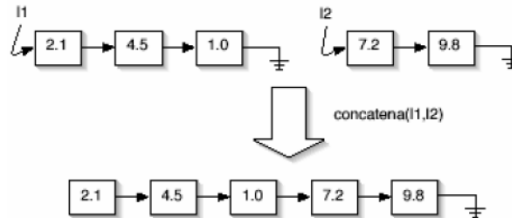
- a) Incluir um elemento no final de uma lista.
- b) Concatenar duas listas.
- c) Liberar todos os nós de uma lista.
- d) Inverter uma lista de modo que o último elemento se torne o primeiro, e assim por diante.
- e) Eliminar o último elemento de uma lista.
- f) Eliminar o *e-nésimo* elemento de uma lista.
- g) Combinar duas listas ordenadas numa única lista ordenada.
- h) Formar uma lista contendo a união dos elementos de duas listas.
- i) Formar uma lista contendo a intersecção dos elementos de duas listas.
- j) Inserir um elemento depois do *e-nésimo* elemento de uma lista.
- k) Eliminar o segundo elemento de uma lista.
- l) Colocar os elementos de uma lista em ordem ascendente.
- m) Retornar a soma dos números inteiros de uma lista.
- n) Retornar a média dos números inteiros de uma lista.
- o) Retornar o número de elementos numa lista.
- p) Retornar a quantidade de elementos da lista que possuem valor ímpar.
- q) Retornar a quantidade de elementos da lista que possuem valor par.
- r) Deslocar um nó *n* posições à frente numa lista.

- s) Criar uma cópia de uma lista.
 - t) Retornar o maior valor de uma lista de inteiros.
17. Implemente uma função para inverter a ordem dos elementos de uma lista encadeada, usando uma pilha.
 18. Implemente uma função para dividir uma lista em duas outras listas, uma com elementos de chaves pares e a outra com elementos de chaves ímpares.
 19. Implemente uma função para verificar se uma lista linear encadeada está ordenada.
 20. Implementar uma função que retorna o valor do primeiro elemento. Se a lista estiver vazia, retornar -1.
 21. Implementar uma função que retorna o valor do último elemento. Se a lista estiver vazia, retornar -1.
 22. Implementar uma função que retorna o endereço do último elemento. Se a lista estiver vazia, retornar NULL.
 23. Implementar uma função que recebe como entrada um vetor com n números inteiros e insere os números em uma lista encadeada.


```
Lista *addvetor(Lista *l, int *vetor, int n);
```
 24. Faça um programa que receba 20 números inteiros e armazene os pares em uma lista simplesmente encadeada não ordenada e os ímpares em uma segunda lista simplesmente encadeada e não ordenada. Posteriormente, o programa deverá montar uma terceira lista, duplamente encadeada e ordenada de forma crescente, com os números das duas listas anteriores. Para finalizar, o programa deve mostrar todos os números da terceira lista na ordem crescente e decrescente.
 25. Implemente uma função para inserir novos elementos no final de uma lista duplamente encadeada.
 26. Escreva uma função que receba uma lista duplamente encadeada e retorne ela invertida.
 27. Implemente uma função para imprimir os elementos de uma lista duplamente encadeada na ordem inversa.
 28. Implemente uma função para inserir elementos em uma lista encadeada circular.
 29. Implemente uma função que receba como entrada uma *lista simplesmente encadeada* de inteiros e um número inteiro positivo n . A função deverá imprimir o elemento da posição n , $n-1$ e $n+1$. Se não houver algum dos elementos a função deverá imprimir o valor -1.
 30. Implemente uma função que receba como entrada uma *lista duplamente encadeada* de inteiros e um número inteiro positivo n . A função deverá imprimir o elemento da posição n , $n-1$ e $n+1$. Se não houver algum dos elementos a função deverá imprimir o valor -1.
 31. Implemente uma função que receba como entrada uma *lista encadeada circular* de inteiros e um número inteiro positivo n . A função deverá imprimir o elemento da posição n , $n-1$ e $n+1$. Se não houver algum dos elementos a função deverá imprimir o valor -1.
 32. Uma estrutura de dados em lista duplamente encadeada permite na cadeia movimentos para

- a) frente, apenas.
- b) trás, apenas.
- c) cima e para baixo ou para frente e para trás.
- d) cima e para baixo, apenas.
- e) frente e para trás, apenas.

33. Considere uma estrutura de lista encadeada que armazena valores reais. Implemente uma função em C que receba como entrada duas listas encadeadas l1 e l2, concatene a lista l2 no final de l1, conforme ilustra a figura abaixo:



A função deve retornar a lista resultante da concatenação, obedecendo ao protótipo:

```
Lista* concatena (Lista *l1, Lista *l2);
```

Observe que l1 e/ou l2 podem ser listas vazias.

34. Acerca de Estrutura de dados, assinale "V" para Verdadeiro e "F" para Falso:

- () Na lista ligada, em cada nó possui um ou mais campos de informação e um apontador para o próximo nó da lista.
- () Filas são estruturas de dados em que inserções são feitas no início e remoções são feitas no final das filas.
- () Na lista ligada circular o último elemento aponta para o primeiro.
- () Numa lista singularmente encadeada, para acessar o último nodo é necessário partir do primeiro e ir seguindo os campos de ligação até chegar ao final da lista.
- () Uma lista é uma coleção de elementos do mesmo tipo dispostos linearmente, que podem ou não seguir determinada organização. As listas podem ser dos seguintes tipos: de encadeamento simples, duplamente encadeadas e ordenadas.
- () Em uma lista circular duplamente encadeada, cada nó aponta para dois outros nós da lista, um anterior e um posterior.
- () As pilhas são listas encadeadas cujos elementos são retirados e acrescentados sempre ao final, enquanto as filas são listas encadeadas cujos elementos são retirados e acrescentados sempre no início.

35. Considerando uma lista de números inteiros, implemente uma função que receba como parâmetro uma lista encadeada e um valor inteiro n e divida a lista em duas, de tal forma que a segunda lista comece no primeiro nó logo após a primeira ocorrência de n na lista original. Essa função deve obedecer ao protótipo:

```
Lista *divide (Lista *l, int n);
```

A função deve retornar um ponteiro para a segunda subdivisão da lista original, enquanto l deve continuar apontando para o primeiro elemento da primeira subdivisão da lista. Se o elemento de valor n não for encontrado na lista, deve retornar NULL e manter a lista l inalterada.

36. Escreva uma função para inserir um elemento antes do n-ésimo elemento de uma lista simplesmente encadeada.

37. Escreva uma função para inserir um elemento antes do n -ésimo elemento de uma lista duplamente encadeada.
38. Escreva uma função que recebe como parâmetro uma lista simplesmente encadeada e um número inteiro n . Caso o número exista na lista ele deve ser transferido para o início da lista.
39. Escreva uma função que recebe como parâmetro uma lista simplesmente encadeada e um número inteiro n . Caso o número exista na lista ele deve ser transferido para o final da lista.
40. Escreva um programa que armazena n valores da sequência de Fibonacci em uma lista simplesmente encadeada. O primeiro elemento da lista deve ser o número 0 e o segundo o número 1.
41. Escreva uma função que tenha como parâmetros ponteiros para duas listas. A função deve retornar um ponteiro para uma terceira lista, que é o resultado da união das duas (isto é, os elementos repetidos devem ser considerados apenas uma vez).
42. Escreva uma função que tenha como parâmetros ponteiros para duas listas. A função deve retornar um ponteiro para uma terceira lista, que é o resultado da intersecção das duas (isto é, deve retornar apenas os elementos presentes em ambas).
43. Considerando uma lista de números inteiros, implemente uma função que receba como parâmetro uma lista encadeada e um valor inteiro positivo n e divida a lista em duas, de tal forma que a segunda lista comece a partir do n -ésimo elemento da lista original. Essa função deve obedecer ao protótipo:

```
Lista *separa (Lista *l, int n);
```

A função deve retornar um ponteiro para a segunda subdivisão da lista original, enquanto l deve continuar apontando para o primeiro elemento da primeira subdivisão da lista.

44. Desenvolva uma função que, dados uma lista simplesmente encadeada e um número inteiro positivo n , remova da lista seus n primeiros nós e retorne a lista resultante. Caso n seja maior do que o tamanho da lista, todos os seus nós devem ser removidos e a lista resultante deve ser uma lista vazia. Essa função deve possuir o seguinte protótipo:

```
Lista *remove_elementos (Lista *l, int n);
```

45. Implemente uma função em C que receba como parâmetro um vetor com n números inteiros e constrói uma lista simplesmente encadeada armazenando os elementos do vetor na lista. Por exemplo, se receber como parâmetro o vetor $v[5] = \{1, 2, 3, 4, 5\}$, a função deve retornar uma lista cujo primeiro elemento tem a informação 1, o segundo a informação 2, e assim por diante. Se o vetor tiver zero elementos, a função deve retornar uma lista vazia. O protótipo da função é dado por:

```
Lista *constroi (int n, int *v);
```

46. Desenvolva uma função em C que receba como parâmetro uma lista encadeada e um número inteiro positivo n e remova da lista todos os nós que possuam valor igual a n . A função deverá retornar a nova lista resultante.

```
Lista* RemoveValor(Lista *l, int n);
```

47. Escreva uma função que recebe como parâmetro uma lista simplesmente encadeada e um número inteiro n e retorne o número de nós da lista que possuem valores menores que n .

```
int Menores(Lista *l, int n);
```

48. Escreva uma função que recebe como parâmetro uma lista simplesmente encadeada e um número inteiro n e retorne uma lista com os valores menores que n .

```
Lista* ListaMenores(Lista *l, int n);
```

49. Escreva uma função que retorne um ponteiro para o último elemento de uma lista simplesmente encadeada.
50. Escreva uma função que retorne um ponteiro para o último elemento de uma lista circular simplesmente encadeada.
51. Escreva uma função que retorne o valor do último elemento de uma lista circular simplesmente encadeada de inteiros.
52. Escreva uma função que retorne o número de elementos de uma lista circular simplesmente encadeada.
53. Considere uma lista encadeada de inteiros. Implemente uma função para retornar o número de nós da lista que possuem um número primo armazenado.
54. Escreva uma função em C que, dada uma lista l qualquer, retorne, numa lista de saída, os elementos ímpares e os elementos pares da lista l , na ordem em que os elementos aparecem em l . Ao final da execução desta função, a lista resultante terá todos os elementos da lista l (primeiro os ímpares, depois os pares, na ordem em que eles aparecem em l), e a lista l continuará contendo os seus elementos. O protótipo da função é o seguinte: `TLista* lparimpar(TLista *l)`.